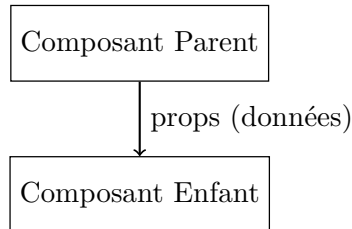


2 Props : Communication Parent-Enfant

2.1 Qu'est-ce que les Props ?

Props = Properties (propriétés)

Les props permettent de passer des données d'un composant parent vers un composant enfant.



Analogie avec fonctions JavaScript :

Listing 2 – Fonction classique vs Composant

```
// Fonction classique
function greet(name, age) { // Parametres
  return `Bonjour ${name}, ${age} ans`;
}
greet("Ahmed", 20); // Arguments

// Composant React
function UserCard(props) { // Props = objet
  return <div>Bonjour {props.name}, {props.age} ans</div>;
}
<UserCard name="Ahmed" age={20} /> // Props passees
```

2.2 Caractéristiques des Props

3 règles essentielles :

1. Les props sont **read-only** (immuables dans l'enfant)
2. Les props passent du parent vers l'enfant (flux unidirectionnel)
3. Les props peuvent contenir n'importe quel type de données

2.3 Syntaxe des Props

2.3.1 Accès via props.nomProp

Listing 3 – Syntaxe de base

```
function ProductCard(props) {
  return (
    <div className="card">
      <h3>{props.name}</h3>
      <p>Prix : {props.price} DH</p>
      <p>Stock : {props.stock}</p>
    </div>
  );
}

// Utilisation
<ProductCard name="Laptop" price={5000} stock={10} />
```

2.3.2 Destructuring (Recommandé)

Le destructuring rend le code plus lisible :

Listing 4 – Avec destructuring

```
// Plus propre et concis
function ProductCard({ name, price, stock }) {
  return (
    <div className="card">
      <h3>{name}</h3>
      <p>Prix : {price} DH</p>
      <p>Stock : {stock}</p>
    </div>
  );
}
```

Avec valeurs par défaut :

Listing 5 – Props avec defaults

```
function ProductCard({ name, price = 0, stock = 0 }) {
  // Si price ou stock non fournis, valeur par défaut utilisée
  return (
    <div className="card">
      <h3>{name}</h3>
      <p>Prix : {price} DH</p>
      <p>Stock : {stock}</p>
    </div>
  );
}

// Sans price ni stock
<ProductCard name="Produit test" />
// Affichera : Prix : 0 DH, Stock : 0
```

2.4 Types de Props

Les props peuvent contenir différents types de données :

Listing 6 – Tous les types de props

```
<ProductCard
  // String (guillemets simples/doubles)
  name="Laptop"

  // Number (accolades obligatoires)
  price={5000}
  stock={10}

  // Boolean (accolades)
  inStock={true}
  featured={false}

  // Array (accolades)
  tags={['nouveau', 'promo', 'populaire']}

  // Object (accolades)
  details={{
    brand: 'HP',
    year: 2024,
    warranty: '2 ans'
  }}

  // Function (accolades)
  onBuy={() => console.log('Achete!')}
/>
```

△ Important

Règle syntaxe :

- String = guillemets : `name="Laptop"`
- Tout le reste = accolades : `price={5000}`

2.5 Exemple Complet

Listing 7 – Exemple realiste ProductCard

```
// Composant enfant
function ProductCard({ name, price, image, inStock, category }) {
  return (
    <div className="product-card">
      <img src={image} alt={name} />
      <span className="category">{category}</span>
      <h3>{name}</h3>
      <p className="price">{price} DH</p>
      {inStock ? (
        <span className="badge green">En stock</span>
      ) : (
        <span className="badge red">Rupture</span>
      )}
      <button>Ajouter au panier</button>
    </div>
  );
}
```

```

}

// Composant parent
function App() {
  return (
    <div className="app">
      <h1>Nos Produits</h1>
      <div className="products-grid">
        <ProductCard
          name="Tajine traditionnel"
          price={250}
          image="/images/tajine.jpg"
          inStock={true}
          category="Cuisine"
        />
        <ProductCard
          name="Babouches cuir"
          price={180}
          image="/images/babouches.jpg"
          inStock={false}
          category="Mode"
        />
      </div>
    </div>
  );
}

```

Avantages de cette approche :

- **Réutilisabilité** : Même composant, données différentes
- **Maintenabilité** : Modifier ProductCard une seule fois
- **Lisibilité** : Interface claire du composant

△ Important

Erreur interdite : Ne JAMAIS modifier les props

```

function ProductCard({ price }) {
  // INTERDIT - Props sont read-only
  price = price * 2;

  // OK - Creer nouvelle variable
  const discountedPrice = price * 0.8;

  return <p>Prix: {discountedPrice} DH</p>;
}

```